

HW1CSE105W25: Homework assignment 1

CSE105W25

Due: January 16th at 5pm, via Gradescope

In this assignment,

You will practice reading and applying the definitions of alphabets, strings, languages, Kleene star, and regular expressions. You will use regular expressions and relate them to languages.

Resources: To review the topics for this assignment, see the class material from Weeks 0 and 1 and Review Quiz 1. We will post frequently asked questions and our answers to them in a pinned Piazza post.

Reading and extra practice problems: Sipser Section 0, 1.3. Chapter 0 exercises 0.1, 0.2, 0.3, 0.5, 0.6, 0.9. Chapter 1 exercises 1.19, 1.23.

For all HW assignments: Weekly homework may be done individually or in groups of up to 3 students. You may switch HW partners for different HW assignments. Please ensure your name(s) and PID(s) are clearly visible on the first page of your homework submission and then upload the PDF to Gradescope. If working in a group, submit only one submission per group: one partner uploads the submission through their Gradescope account and then adds the other group member(s) to the Gradescope submission by selecting their name(s) in the “Add Group Members” dialog box. You will need to re-add your group member(s) every time you resubmit a new version of your assignment. Each homework question will be graded either for correctness (including clear and precise explanations and justifications of all answers) or fair effort completeness. On the “graded for correctness” questions, you may only collaborate with CSE 105 students in your group; if your group has questions about a problem, you may ask in drop-in help hours or post a private post (visible only to the Instructors) on Piazza. On the “graded for completeness” questions, you may collaborate with all other CSE 105 students this quarter, and you may make public posts about these questions on Piazza.

All submitted homework for this class must be typed. You can use a word processing editor if you like (Microsoft Word, Open Office, Notepad, Vim, Google Docs, etc.) but you might find it useful to take this opportunity to learn LaTeX. LaTeX is a markup language used widely in computer science and mathematics. The homework assignments are typed using LaTeX and you can use the source files as templates for typesetting your solutions. To generate state diagrams of

machines, you can (1) use the LaTeX tikzpicture environment (see templates in the class notes), or (2) use the software tools Flap.js or JFLAP described in the class syllabus (and include a screenshot in your PDF), or (3) you can carefully and clearly hand-draw the diagram and take a picture and include it in your PDF. We recommend that you submit early drafts to Gradescope so that in case of any technical difficulties, at least some of your work is present. You may update your submission as many times as you'd like up to the deadline.

Integrity reminders

- Problems should be solved together, not divided up between the partners. The homework is designed to give you practice with the main concepts and techniques of the course, while getting to know and learn from your classmates.
- On the “graded for correctness” questions, you may only collaborate with CSE 105 students in your group. You may ask questions about the homework in office hours (of the instructor, TAs, and/or tutors) and on Piazza (as private notes viewable only to the Instructors). You *cannot* use any online resources about the course content other than the class material from this quarter – this is primarily to ensure that we all use consistent notation and definitions (aligned with the textbook) and also to protect the learning experience you will have when the ‘aha’ moments of solving the problem authentically happen.
- Do not share written solutions or partial solutions for homework with other students in the class who are not in your group. Doing so would dilute their learning experience and detract from their success in the class.

You will submit this assignment via Gradescope (<https://www.gradescope.com>) in the assignment called “hw1CSE105W25”.

Assigned questions

1. Strings and languages: finding examples and edge cases (12 points):

- (a) (*Graded for completeness*)¹ Give five (different) example alphabets that are meaningful or useful to you in some way. Specify them formally, either with roster notation (which means listing all and only distinct elements between { and } and separated by commas) or with another approach to precisely define all and only the elements of the alphabet.
- (b) (*Graded for completeness*) Give an example of a finite set over an alphabet and an infinite set over an alphabet. You get to choose the alphabet, and you get to choose the sets.

¹This means you will get full credit so long as your submission demonstrates honest effort to answer the question. You will not be penalized for incorrect answers. To demonstrate your honest effort in answering the question, we expect you to include your attempt to answer *each* part of the question. If you get stuck with your attempt, you can still demonstrate your effort by explaining where you got stuck and what you did to try to get unstuck.

The goal is to practice communicating your choices and definitions with clear and precise notation. One habit that will be useful (for this course, and beyond), is to think of your response for each question as a well-formed paragraph: include all the information that is relevant so that your solution is self-contained, and so that each sentence is grammatically constructed.

- (c) (*Graded for correctness*)² Define an alphabet Σ_1 and an alphabet Σ_2 and a language L_1 over Σ_1 that is also a language over Σ_2 and a language L_2 over Σ_2 that is **not** a language over Σ_1 . A complete and correct answer will use clear and precise notation (consistent with the textbook and class notes) and will include a description of why the given example L_1 is a language over both Σ_1 and Σ_2 and a description of why the given example L_2 is a language over Σ_2 and not over Σ_1 .

2. Regular expressions (20 points):

- (a) (*Graded for completeness*) Give three regular expressions that all describe the set of all strings over $\{a, b\}$ that have odd length. Ungraded bonus challenge: Make the expressions as different as possible!
- (b) (*Graded for completeness*) A friend tells you that each regular expression that has a Kleene star ($*$) describes an infinite language. Are they right? Either help them justify their claim or give a counterexample to disprove it and explain your counterexample.
- (c) (*Graded for correctness*) For this question, the alphabet is $\{a, b, c\}$. A friend is trying to design a regular expression that describes the set of all strings over this alphabet that end in c . Classify each of the following attempts as
- Correct. Explain why.
 - Error Type 1: Incorrect, because (even though each string that is in the language described by the regular expression ends in c) there is a string that ends in c that is not in the language described by the regular expression. Give this example string and explain why it proves we're in this case.
 - Error Type 2: Incorrect, because (even though each string that ends in c is in the language described by the regular expression), there is a string in the language described by the regular expression that does not end in c . Give this example string and explain why it proves we're in this case.
 - Error Type 3: Incorrect, because there are two counterexample strings, one which is a string that ends in c that is not in the language described by the regular expression and one which is in the language described by the regular expression but does not end in c . Give both example strings and describe why each has the given property.

²This means your solution will be evaluated not only on the correctness of your answers, but on your ability to present your ideas clearly and logically. You should explain how you arrived at your conclusions, using mathematically sound reasoning. Whether you use formal proof techniques or write a more informal argument for why something is true, your answers should always be well-supported. Your goal should be to convince the reader that your results and methods are sound.

Worked example for reference: Consider the regular expression $(a \cup b \cup c)^*$. This regular expression has **Error Type 2** because it describes the set of all strings over $\{a, b, c\}$, so even though each string that ends in c is in this language, there is an example, say ab that is a string in the language described by the regular expression (because we consider the string formed as a result of the Kleene star operation which has 2 slots and where the first slot matches the a in $a \cup b \cup c$ and the second slot matches b in $a \cup b \cup c$) but does not end in c (it ends in b).

i. The regular expression is

$$(a \cup b)^* \circ c$$

ii. The regular expression is

$$(a \circ b \circ c)^*$$

iii. The regular expression is

$$a^*c \cup b^*c \cup c^*c$$

3. Functions over languages (18 points):

For each language L over an alphabet Σ , we have the associated sets of strings (also over Σ)

$$L^* = \{w_1 \cdots w_k \mid k \geq 0 \text{ and each } w_i \in L\}$$

and

$$SUBSTRING(L) = \{w \in \Sigma^* \mid \text{there exist } x, y \in \Sigma^* \text{ such that } xwy \in L\}$$

and

$$EXTEND(L) = \{w \in \Sigma^* \mid w = uv \text{ for some strings } u \in L \text{ and } v \in \Sigma^*\}$$

Also, recall the set operations union and intersection: for any sets X and Y

$$X \cup Y = \{w \mid w \in X \text{ or } w \in Y\}$$

$$X \cap Y = \{w \mid w \in X \text{ and } w \in Y\}$$

(a) (*Graded for completeness*) Specify an example language A over $\{0, 1\}$ such that

$$SUBSTRING(A) = EXTEND(A)$$

or explain why there is no such example. A complete solution will include either (1) a precise and clear description of your example language A and a precise and clear description of the result of computing $SUBSTRING(A)$, $EXTEND(A)$ (using the given definitions) to justify this description and to justify the set equality, or (2) a sufficiently general and correct argument why there is no such example, referring back to the relevant definitions.

(b) (*Graded for correctness*) Specify an example language B over $\{0, 1\}$ such that

$$SUBSTRING(B) \cap EXTEND(B) = \{\varepsilon\}$$

and

$$SUBSTRING(B) \cup EXTEND(B) = \{0, 1\}^*$$

or explain why there is no such example. A complete solution will include either (1) a precise and clear description of your example language B and a precise and clear description of the result of computing $SUBSTRING(B)$, $EXTEND(B)$ (using the given definitions) to justify this description and to justify the set equality with $\{\varepsilon\}$ and $\{0, 1\}^*$ (respectively), or (2) a sufficiently general and correct argument why there is no such example, referring back to the relevant definitions.

- (c) (*Graded for correctness*) Specify an example **infinite** language C over $\{0, 1\}$ such that

$$SUBSTRING(C) \neq \{0, 1\}^*$$

and

$$SUBSTRING(C) = C^*$$

or explain why there is no such example. A complete solution will include either (1) a precise and clear description of your example language C and a precise and clear description of the result of computing $SUBSTRING(C)$, C^* (using the given definitions) to justify this description and to justify the set nonequality claims, or (2) a sufficiently general and correct argument why there is no such example, referring back to the relevant definitions.

- (d) (*Graded for correctness*) Specify an example **finite** language D over $\{0, 1\}$ such that

$$EXTEND(D) \neq \{0, 1\}^*$$

and

$$EXTEND(D) = D^*$$

or explain why there is no such example. A complete solution will include either (1) a precise and clear description of your example language D and a precise and clear description of the result of computing $EXTEND(D)$, D^* (using the given definitions) to justify this description and to justify the set nonequality claims, or (2) a sufficiently general and correct argument why there is no such example, referring back to the relevant definitions.

HW2CSE105W25: Homework assignment 2 Due: January 30th at 5pm, via Gradescope

In this assignment,

You will practice designing multiple representations of regular languages and working with general constructions of automata to demonstrate the richness of the class of regular languages.

Resources: To review the topics for this assignment, see the class material from Week 2 and Week 3. We will post frequently asked questions and our answers to them in a pinned Piazza post.

Reading and extra practice problems: Sipser Section 1.1, 1.2, 1.3. Chapter 1 exercises 1.4, 1.5, 1.6, 1.7, 1.8, 1.9, 1.10, 1.11, 1.12, 1.13, 1.14, 1.15, 1.16, 1.17, 1.18, 1.19, 1.20, 1.21, 1.22, 1.23. Chapter 1 problem 1.31, 1.36, 1.37.

For all HW assignments: Weekly homework may be done individually or in groups of up to 3 students. You may switch HW partners for different HW assignments. Please ensure your name(s) and PID(s) are clearly visible on the first page of your homework submission and then upload the PDF to Gradescope. If working in a group, submit only one submission per group: one partner uploads the submission through their Gradescope account and then adds the other group member(s) to the Gradescope submission by selecting their name(s) in the “Add Group Members” dialog box. You will need to re-add your group member(s) every time you resubmit a new version of your assignment. Each homework question will be graded either for correctness (including clear and precise explanations and justifications of all answers) or fair effort completeness. On the “graded for correctness” questions, you may only collaborate with CSE 105 students in your group; if your group has questions about a problem, you may ask in drop-in help hours or post a private post (visible only to the Instructors) on Piazza. On the “graded for completeness” questions, you may collaborate with all other CSE 105 students this quarter, and you may make public posts about these questions on Piazza.

All submitted homework for this class must be typed. You can use a word processing editor if you like (Microsoft Word, Open Office, Notepad, Vim, Google Docs, etc.) but you might find it useful to take this opportunity to learn LaTeX. LaTeX is a markup language used widely in computer science and mathematics. The homework assignments are typed using LaTeX and you can use the source files as templates for typesetting your solutions. To generate state diagrams of machines, you can (1) use the LaTeX tikzpicture environment (see templates in the class notes), or (2) use the software tools Flap.js or JFLAP described in the class syllabus (and include a screenshot in your PDF), or (3) you can carefully and clearly hand-draw the diagram and take a picture and include it in your PDF. We recommend that you submit early drafts to Gradescope so that in case of any technical difficulties, at least some of your work is present. You may update your submission as many times as you’d like up to the deadline.

Integrity reminders

- Problems should be solved together, not divided up between the partners. The homework

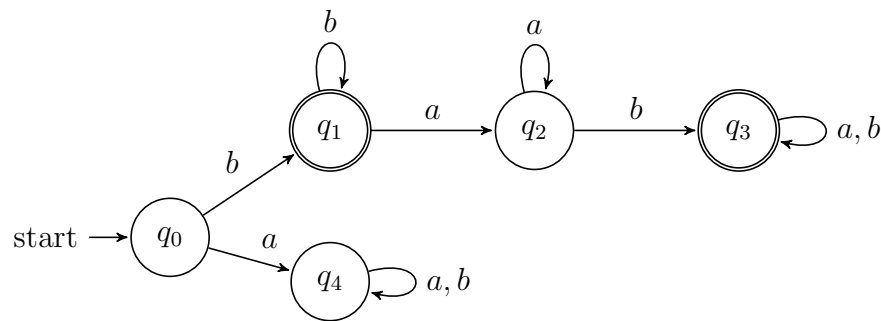
is designed to give you practice with the main concepts and techniques of the course, while getting to know and learn from your classmates.

- On the “graded for correctness” questions, you may only collaborate with CSE 105 students in your group. You may ask questions about the homework in office hours (of the instructor, TAs, and/or tutors) and on Piazza (as private notes viewable only to the Instructors). You *cannot* use any online resources about the course content other than the class material from this quarter – this is primarily to ensure that we all use consistent notation and definitions (aligned with the textbook) and also to protect the learning experience you will have when the ‘aha’ moments of solving the problem authentically happen.
- Do not share written solutions or partial solutions for homework with other students in the class who are not in your group. Doing so would dilute their learning experience and detract from their success in the class.

You will submit this assignment via Gradescope (<https://www.gradescope.com>) in the assignment called “hw2CSE105W25”.

Assigned questions

1. **Finite automata** (10 points): Consider the finite automaton $M = (Q, \Sigma, \delta, q_0, F)$ whose state diagram is depicted below



- (*Graded for completeness*)³ Write the formal definition of this automaton. In other words, give the five defining parameters $Q, \Sigma, \delta, q_0, F$ so that they are consistent with the state diagram of M .
- (*Graded for correctness*)⁴ Give a regular expression R so that $L(R) = L(M)$. In other words, we want a regular expression that describes the language recognized by this finite

³This means you will get full credit so long as your submission demonstrates honest effort to answer the question. You will not be penalized for incorrect answers. To demonstrate your honest effort in answering the question, we expect you to include your attempt to answer **each** part of the question. If you get stuck with your attempt, you can still demonstrate your effort by explaining where you got stuck and what you did to try to get unstuck.

⁴This means your solution will be evaluated not only on the correctness of your answers, but on your ability to present your ideas clearly and logically. You should explain how you arrived at your conclusions, using

automaton. Justify your answer by referring to the definition of the semantics of regular expressions and computations of finite automata. Include an explanation for why each string in $L(R)$ is accepted by the finite automaton *and* for why each string not in $L(R)$ is rejected by the finite automaton.

Ungraded bonus: can you find more than one such regular expression?

- (c) (*Graded for completeness*) Keeping the same set of states Q , input alphabet Σ , same start state q_0 , and same transition function δ , choose a new set of accepting states F_{new1} so that the new finite automaton $M_1 = (Q, \Sigma, \delta, q_0, F_{new1})$ that results recognizes a **proper superset** of $L(M)$, or explain why there is no such example. A complete solution will include either (1) a precise and clear description of your choice of F_{new1} *and* a precise and clear explanation of why every string that is accepted by M is also accepted by M_1 *and* an example of a string that is accepted by M_1 and is rejected by M ; or (2) a sufficiently general and correct argument why there is no such example, referring back to the relevant definitions.
- (d) (*Graded for correctness*) Keeping the same set of states Q , input alphabet Σ , same start state q_0 , and same transition function δ , choose a new set of accepting states F_{new2} so that the new finite automaton $M_2 = (Q, \Sigma, \delta, q_0, F_{new2})$ that results recognizes a **nonempty proper subset** of $L(M)$, or explain why there is no such example. A complete solution will include either (1) a precise and clear description of your choice of F_{new2} *and* an example string accepted by M_2 *and* a precise and clear explanation of why every string that is accepted by M_2 is also accepted by M *and* an example of a string that is accepted by M and is rejected by M_2 ; or (2) a sufficiently general and correct argument why there is no such example, referring back to the relevant definitions.

2. **Automata design** (12 points): As background to this question, recall that integers can be represented using base b expansions, for any convenient choice of base b . The precise definition is: for b an integer greater than 1 and n a positive integer, the **base b expansion of n** is defined to be

$$(a_{k-1} \cdots a_1 a_0)_b$$

where k is a positive integer, $a_0, a_1, \dots, a_{k-1}$ are nonnegative integers less than b , $a_{k-1} \neq 0$, and

$$n = \sum_{i=0}^{k-1} a_i b^i$$

Notice: *The base b expansion of a positive integer n is a string over the alphabet $\{x \in \mathbb{Z} \mid 0 \leq x < b\}$ whose leftmost character is nonzero.*

An important property of base b expansions of integers is that, for each integer b greater than 1, each positive integer $n = (a_{k-1} \cdots a_1 a_0)_b$, and each nonnegative integer a less than b ,

$$bn + a = (a_{k-1} \cdots a_1 a_0 a)_b$$

mathematically sound reasoning. Whether you use formal proof techniques or write a more informal argument for why something is true, your answers should always be well-supported. Your goal should be to convince the reader that your results and methods are sound.

In other words, shifting the base b expansion to the left results in multiplying the integer value by the base. In this question we'll explore building deterministic finite automata that recognize languages that correspond to useful sets of integers.

- (a) (*Graded for completeness*) Design a DFA that recognizes the set of binary (base 2) expansions of positive integers that are powers of 2. A complete solution will include the state diagram of your DFA and a brief justification of your construction by explaining the role each state plays in the machine, as well as a brief justification about how the strings accepted and rejected by the machine connect to the specified language.

Hints: (1) A power of 2 is an integer x that can be written as 2^y for some nonnegative integer y , (2) the DFA should accept the strings 100, 10 and 100000 and should reject the strings 010, 1101, and ε (can you see why?).

- (b) (*Graded for correctness*) Design a DFA that recognizes the set of binary (base 2) expansions of positive integers that are less than 10. Your DFA must use **fewer than ten states**. A complete solution will include the state diagram of your DFA and a brief justification of your construction by explaining the role each state plays in the machine, as well as a brief justification about how the strings accepted and rejected by the machine connect to the specified language.

- (c) (*Graded for completeness*) Find a positive integer B greater than 1 so that there is a DFA that recognizes the set of base B expansions of positive integers that are less than 10 and it uses as few states as possible. A complete solution will include the state diagram of your DFA and a brief justification of your choice of base.

Hint: sometimes rewriting the defining membership condition for a set in different ways helps us find alternate representations of that set.

3. Nondeterminism (15 points): For this question, the alphabet is $\{a, b, c\}$.

- (a) (*Graded for completeness*) Design a NFA that recognizes the language

$$L_1 = \{w \in \{a, b, c\}^* \mid w \text{ starts with } a \text{ and ends with } a\}$$

A complete solution will include the state diagram of your NFA and a brief justification of your construction that explains the role each state plays in the machine, as well as a brief justification about how the strings accepted and rejected by the machine connect to the specified language.

- (b) (*Graded for correctness*) Design a NFA that recognizes the language

$$L_2 = \{w \in \{a, b, c\}^* \mid w \text{ has no consecutive repeated characters}\}$$

For example, the empty string, a , bac , and $abca$ are each elements of this language but aa and abb and $abbc$ are not elements of this language.

A complete solution will include the state diagram of your NFA and a brief justification of your construction that explains the role each state plays in the machine, as well as a brief justification about how the strings accepted and rejected by the machine connect to the specified language.

(c) (*Graded for completeness*) Consider the language

$$L_1 \cup L_2 = \{w \in \{a, b, c\}^* \mid w \text{ starts with } a \text{ and ends with } a \\ \text{or has no consecutive repeated characters}\}$$

Give at least two representations of this language among the following:

- A regular expression that describes $L_1 \cup L_2$
- A DFA that recognizes $L_1 \cup L_2$
- A NFA that recognizes $L_1 \cup L_2$

You can design your automata directly or use the constructions from class and chapter 1 in the book to build these automata from automata for the simpler languages.

A complete solution will include at least two of the representations as well as a brief justification of each construction.

(d) (*Graded for completeness*) Consider the language

$$L_1 \cap L_2 = \{w \in \{a, b, c\}^* \mid w \text{ starts with } a \text{ and ends with } a \\ \text{and has no consecutive repeated characters}\}$$

Give at least two representations of this language among the following:

- A regular expression that describes $L_1 \cap L_2$
- A DFA that recognizes $L_1 \cap L_2$
- A NFA that recognizes $L_1 \cap L_2$

You can design your automata directly or use the constructions from class and chapter 1 in the book to build these automata from automata for the simpler languages.

A complete solution will include at least two of the representations as well as a brief justification of each construction.

4. General constructions (13 points): In this question, you'll practice working with formal general constructions for automata and translating between state diagrams and formal definitions.

Recall the definitions of operations we've talked about that produce new languages from old: for each language L over an alphabet Σ , we have the associated sets of strings (also over Σ)

$$L^* = \{w_1 \cdots w_k \mid k \geq 0 \text{ and each } w_i \in L\}$$

and

$$SUBSTRING(L) = \{w \in \Sigma^* \mid \text{there exist } x, y \in \Sigma^* \text{ such that } xwy \in L\}$$

and

$$EXTEND(L) = \{w \in \Sigma^* \mid w = uv \text{ for some strings } u \in L \text{ and } v \in \Sigma^*\}$$

Also, recall the set operations union and intersection: for any sets X and Y

$$X \cup Y = \{w \mid w \in X \text{ or } w \in Y\}$$

$$X \cap Y = \{w \mid w \in X \text{ and } w \in Y\}$$

Let $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ and $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ be DFA.

For simplicity, assume that $Q_1 \cap Q_2 = \emptyset$ and that $q_0 \notin Q_1 \cup Q_2$.

Consider the following definitions of new automata parameterized by these DFA:

- The NFA $N_\alpha = (Q_1 \cup Q_2 \cup \{q_0\}, \Sigma, \delta_\alpha, q_0, F_1 \cup F_2)$ with the transition function given by

$$\delta_\alpha((q, x)) = \begin{cases} \{q_1, q_2\} & \text{if } q = q_0, x = \varepsilon \\ \emptyset & \text{if } q = q_0, x \in \Sigma \\ \{\delta_1((q, x))\} & \text{if } q \in Q_1, x \in \Sigma \\ \{\delta_2((q, x))\} & \text{if } q \in Q_2, x \in \Sigma \\ \emptyset & \text{if } q \in Q_1 \cup Q_2, x = \varepsilon \end{cases}$$

- The NFA $N_\beta = (Q_1 \times Q_2, \Sigma, \delta_\beta, (q_1, q_2), F_1 \times F_2)$ with the transition function given by

$$\delta_\beta(((r, s), x)) = \{(\delta_1((r, x)), \delta_2((s, x)))\}$$

and

$$\delta_\beta(((r, s), \varepsilon)) = \emptyset$$

for $r \in Q_1, s \in Q_2, x \in \Sigma$.

- The NFA $N_\gamma = (Q_1 \cup \{q_0\}, \Sigma, \delta_\gamma, q_0, \{q \in Q_1 \mid \exists w \in \Sigma^* (\delta_1^*((q, w)) \in F_1)\})$, and

$$\delta_\gamma((q, a)) = \begin{cases} \{\delta_1((q, a))\} & \text{if } q \in Q_1, a \in \Sigma \\ \{q' \in Q_1 \mid \exists w \in \Sigma^* (\delta_1^*((q_1, w)) = q')\} & \text{if } q = q_0, a = \varepsilon \\ \emptyset & \text{if } q = q_0, a \in \Sigma \\ \emptyset & \text{if } q \in Q_1, a = \varepsilon \end{cases}$$

Hint: the notation δ_1^ refers to the iterated transition function.*

- (a) (*Graded for correctness*) Illustrate the construction of N_α by defining a specific pair of example DFAs M_1 and M_2 and applying the construction above to create the new NFA N_α . Your example DFA should

- Have the same input alphabet as each other,
- Each have exactly three states (all reachable from the respective start state),
- Accept at least one string and reject at least one string,
- Recognize different languages from one another, and
- Not have any states labelled q_0 , and
- Not share any state labels.

Apply the construction above to create the new NFA. A complete submission will include the state diagrams of your example DFA M_1 and M_2 and the state diagram of the NFA N_α resulting from this construction and a precise and clear description of $L(M_1)$ and $L(M_2)$ and $L(N_\alpha)$, justified by explaining the role each state plays in the machine, as well as a brief justification about how the strings accepted and rejected by the machine connect to the language.

- (b) (*Graded for correctness*) Illustrate the construction of N_β by defining a specific pair of example DFAs M_1 and M_2 and applying the construction above to create the new NFA N_β . Your example DFA should

- Have the same input alphabet as each other,
- Each have exactly two states (all reachable from the respective start state),
- Accept at least one string and reject at least one string,
- Recognize different languages from one another, and
- Not have any states labelled q_0 , and
- Not share any state labels.

Apply the construction above to create the new NFA. A complete submission will include the state diagrams of your example DFA M_1 and M_2 and the state diagram of the NFA N_β resulting from this construction and a precise and clear description of $L(M_1)$ and $L(M_2)$ and $L(N_\beta)$, justified by explaining the role each state plays in the machine, as well as a brief justification about how the strings accepted and rejected by the machine connect to the language.

- (c) (*Graded for correctness*) Illustrate the construction of N_γ by defining a specific example DFA M_1 and applying the construction above to create the new NFA N_γ . Your example DFA should

- Have exactly four states (all reachable from the respective start state),
- Accept at least one string and reject at least one string,
- Not have any states labelled q_0 .

Apply the construction above to create the new NFA. A complete submission will include the state diagram of your example DFA M_1 and the state diagram of the NFA N_γ resulting from this construction and a precise and clear description of $L(M_1)$ and $L(N_\gamma)$, justified by explaining the role each state plays in the machine, as well as a brief justification about how the strings accepted and rejected by the machine connect to the language.

- (d) (*Graded for completeness*) If possible, associate each construction above with one of the operations whose definitions we recalled at the start of the question. For example, is it the case that (for all choices of DFA M_1 and M_2) $L(N_\alpha) = L(M_1) \cup L(M_2)$? or $L(N_\alpha) = L(M_1) \cap L(M_2)$? etc.

A complete solution will consider each of the constructions $N_\alpha, N_\beta, N_\gamma$ in turn, and for each, either name the operation that's associated with the construction (and explain why) or explain why none of the operations mentioned is associated with the construction.

HW3CSE105W25: Homework assignment 3 Due: February 6th at 5pm, via Gradescope

In this assignment,

You will demonstrate the richness of the class of regular languages, as well as its boundaries, and explore push-down automata and their design.

Resources: To review the topics for this assignment, see the class material from Week 3 and Week 4. We will post frequently asked questions and our answers to them in a pinned Piazza post.

Reading and extra practice problems: Sipser Chapter 1 and Section 2.2. Chapter 1 exercises 1.28, 1.29, 1.30. Chapter 1 problem 1.53, 1.54, 1.55. Chapter 2 exercises 2.7, 2.10.

For all HW assignments: Weekly homework may be done individually or in groups of up to 3 students. You may switch HW partners for different HW assignments. Please ensure your name(s) and PID(s) are clearly visible on the first page of your homework submission and then upload the PDF to Gradescope. If working in a group, submit only one submission per group: one partner uploads the submission through their Gradescope account and then adds the other group member(s) to the Gradescope submission by selecting their name(s) in the “Add Group Members” dialog box. You will need to re-add your group member(s) every time you resubmit a new version of your assignment. Each homework question will be graded either for correctness (including clear and precise explanations and justifications of all answers) or fair effort completeness. On the “graded for correctness” questions, you may only collaborate with CSE 105 students in your group; if your group has questions about a problem, you may ask in drop-in help hours or post a private post (visible only to the Instructors) on Piazza. On the “graded for completeness” questions, you may collaborate with all other CSE 105 students this quarter, and you may make public posts about these questions on Piazza.

All submitted homework for this class must be typed. You can use a word processing editor if you like (Microsoft Word, Open Office, Notepad, Vim, Google Docs, etc.) but you might find it useful to take this opportunity to learn LaTeX. LaTeX is a markup language used widely in computer science and mathematics. The homework assignments are typed using LaTeX and you can use the source files as templates for typesetting your solutions. To generate state diagrams of machines, you can (1) use the LaTeX tikzpicture environment (see templates in the class notes), or (2) use the software tools Flap.js or JFLAP described in the class syllabus (and include a screenshot in your PDF), or (3) you can carefully and clearly hand-draw the diagram and take a picture and include it in your PDF. We recommend that you submit early drafts to Gradescope so that in case of any technical difficulties, at least some of your work is present. You may update your submission as many times as you’d like up to the deadline.

Integrity reminders

- Problems should be solved together, not divided up between the partners. The homework is designed to give you practice with the main concepts and techniques of the course, while

getting to know and learn from your classmates.

- On the “graded for correctness” questions, you may only collaborate with CSE 105 students in your group. You may ask questions about the homework in office hours (of the instructor, TAs, and/or tutors) and on Piazza (as private notes viewable only to the Instructors). You *cannot* use any online resources about the course content other than the class material from this quarter – this is primarily to ensure that we all use consistent notation and definitions (aligned with the textbook) and also to protect the learning experience you will have when the ‘aha’ moments of solving the problem authentically happen.
- Do not share written solutions or partial solutions for homework with other students in the class who are not in your group. Doing so would dilute their learning experience and detract from their success in the class.

You will submit this assignment via Gradescope (<https://www.gradescope.com>) in the assignment called “hw3CSE105W25”.

Assigned questions

1. **Static analysis** (10 points): In software engineering, static analysis is an approach to debugging and testing where the properties of a piece of code are inferred without actually running it. In the context of finite automata, we can think of static analysis as the process of inferring properties of the language recognized by a finite automaton from properties of the graph underlying its state diagram. The Pumping Lemma is one example of static analysis. In this question, you’ll explore other examples of how properties of the graph underlying the state diagram of a machine can give us information about the language recognized by the machine.

- (a) (*Graded for completeness*)⁵ Suppose you are given an NFA N_0 over an alphabet Σ and each accepting state in N_0 is not reachable from the start state of N_0 . What can you conclude about the language of the NFA?
- (b) (*Graded for correctness*)⁶ Prove or disprove: For any alphabet Σ and any DFA M over Σ , if every state in M is accepting then $L(M) = \Sigma^*$. A complete answer will clearly indicate whether the statement is true or false and then will justify with a complete and correct argument.

⁵This means you will get full credit so long as your submission demonstrates honest effort to answer the question. You will not be penalized for incorrect answers. To demonstrate your honest effort in answering the question, we expect you to include your attempt to answer **each** part of the question. If you get stuck with your attempt, you can still demonstrate your effort by explaining where you got stuck and what you did to try to get unstuck.

⁶This means your solution will be evaluated not only on the correctness of your answers, but on your ability to present your ideas clearly and logically. You should explain how you arrived at your conclusions, using mathematically sound reasoning. Whether you use formal proof techniques or write a more informal argument for why something is true, your answers should always be well-supported. Your goal should be to convince the reader that your results and methods are sound.

- (c) (*Graded for correctness*) Prove or disprove: For any alphabet Σ and any NFA N over Σ , if every state in N is accepting then $L(N) = \Sigma^*$. A complete answer will clearly indicate whether the statement is true or false and then will justify with a complete and correct argument.

2. Multiple representations (12 points):

- (a) Consider the language $A_1 = \{uw \mid u \text{ and } w \text{ are strings over } \{0, 1\} \text{ and have the same length}\}$ and the following argument.

“Proof” that A_1 is not regular using the Pumping Lemma: Let p be an arbitrary positive integer. We will show that p is not a pumping length for A_1 .

Choose s to be the string $1^p 0^p$, which is in A_1 because we can choose $u = 1^p$ and $w = 0^p$ which each have length p . Since s is in A_1 and has length greater than or equal to p , if p were to be a pumping length for A_1 , s ought to be pump’able. That is, there should be a way of dividing s into parts x, y, z where $s = xyz$, $|y| > 0$, $|xy| \leq p$, and for each $i \geq 0$, $xy^i z \in A_1$. Suppose x, y, z are such that $s = xyz$, $|y| > 0$ and $|xy| \leq p$. Since the first p letters of s are all 1 and $|xy| \leq p$, we know that x and y are made up of all 1s. If we let $i = 2$, we get a string $xy^2 z$ that is not in A_1 because repeating y twice adds 1s to u but not to w , and strings in A_1 are required to have u and w be the same length. Thus, s is not pumpable (even though it should have been if p were to be a pumping length) and so p is not a pumping length for A_1 . Since p was arbitrary, we have demonstrated that A_1 has no pumping length. By the Pumping Lemma, this implies that A_1 is nonregular.

- (*Graded for completeness*) Find the (first and/or most significant) logical error in the “proof” above and describe why it’s wrong.
 - (*Graded for completeness*) Prove that the set A_1 is actually regular (by finding a regular expression that describes it or a DFA/NFA that recognizes it, and justifying why) **or** fix the proof so that it is logically sound.
- (b) Consider the language $A_2 = \{u1w \mid u \text{ and } w \text{ are strings over } \{0, 1\} \text{ and have the same length}\}$ and the following argument.

“Proof” that A_2 is not regular using the Pumping Lemma: Let p be an arbitrary positive integer. We will show that p is not a pumping length for A_2 .

Choose s to be the string $1^{p+1} 0^p$, which is in A_2 because we can choose $u = 1^p$ and $w = 0^p$ which each have length p . Since s is in A_2 and has length greater than or equal to p , if p were to be a pumping length for A_2 , s ought to be pump’able. That is, there should be a way of dividing s into parts x, y, z where $s = xyz$, $|y| > 0$, $|xy| \leq p$, and for each $i \geq 0$, $xy^i z \in A_2$. When $x = \varepsilon$ and $y = 1^{p+1}$ and $z = 0^p$, we have satisfied that $s = xyz$, $|y| > 0$ (because p is positive) and $|xy| \leq p$. If we let $i = 0$, we get the string $xy^0 z = 0^p$ that is not in A_2 because its middle symbol is a 0, not a 1. Thus, s is not pumpable (even though it should have been if p were to be a pumping length) and so p is not a pumping length for A_2 . Since p was arbitrary, we have demonstrated that A_2 has no pumping length. By the Pumping Lemma, this implies that A_2 is nonregular.

- i. (*Graded for completeness*) Find the (first and/or most significant) logical error in the “proof” above and describe why it’s wrong.
- ii. (*Graded for completeness*) Prove that the set A_2 is actually regular (by finding a regular expression that describes it or a DFA/NFA that recognizes it, and justifying why) **or** fix the proof so that it is logically sound.

3. Pumping (10 points):

- (a) (*Graded for correctness*) Give an example of a language over the alphabet $\{a, b\}$ that has cardinality 3 and for which 5 is a pumping length and 4 is not a pumping length. Is this language regular? A complete solution will give (1) a clear and precise description of the language, (2) a justification for why 5 is a pumping length, (3) a justification for why 4 is not a pumping length, (4) a correct and justified answer to whether the language is regular.
- (b) (*Graded for completeness*) In class and in the reading so far, we’ve seen the following examples of nonregular sets:

$$\begin{array}{lll}
 \{0^n 1^n \mid n \geq 0\} & \{0^n 1^m \mid 0 \leq m \leq n\} & \{0^n 1^m 0^n \mid n, m \geq 0\} \\
 \{0^n 1^n \mid n \geq 2\} & \{0^i 1^{2i} \mid 0 \leq i\} & \{w \in \{0, 1\}^* \mid w = w^R\} \\
 \{0^n 1^m \mid 0 \leq n \leq m\} & \{0^i 1^{i+1} \mid 0 \leq i\} & \{ww^R \mid w \in \{0, 1\}^*\}
 \end{array}$$

Modify one of these sets in some way and use the Pumping Lemma to prove that the resulting set is still nonregular.

4. Regular and nonregular languages (12 points): In Week 2’s review quiz, we saw the definition that a set X is said to be **closed under an operation** if, for any elements in X , applying to them gives an element in X . For example, the set of integers is closed under multiplication because if we take any two integers, their product is also an integer .

Recall the definitions of operations we’ve talked about that produce new languages from old: for each language L over an alphabet Σ , we have the associated sets of strings (also over Σ)

$$L^* = \{w_1 \cdots w_k \mid k \geq 0 \text{ and each } w_i \in L\}$$

and

$$\text{SUBSTRING}(L) = \{w \in \Sigma^* \mid \text{there exist } x, y \in \Sigma^* \text{ such that } xwy \in L\}$$

and

$$\text{EXTEND}(L) = \{w \in \Sigma^* \mid w = uv \text{ for some strings } u \in L \text{ and } v \in \Sigma^*\}$$

Also, recall the set operations union and intersection: for any sets X and Y

$$X \cup Y = \{w \mid w \in X \text{ or } w \in Y\}$$

$$X \cap Y = \{w \mid w \in X \text{ and } w \in Y\}$$

- (a) (*Graded for correctness*) Use the general constructions that we developed to prove the closure of the class of regular languages under various operations to produce the state diagram of a NFA that recognizes the language

$$(SUBSTRING(\{0, 01, 111\}))^*$$

Hint: Question 4 from Homework 2 might be helpful.

For full credit, submit (1) a state diagram of an NFA that recognizes $\{0, 01, 111\}$, (2) a state diagram of an NFA that recognizes $SUBSTRING(\{0, 01, 111\})$, and (3) a state diagram of an NFA that recognizes $(SUBSTRING(\{0, 01, 111\}))^*$, and a brief justification of each state diagram that references the language being recognized or the general constructions being used.

- (b) (*Graded for completeness*) Prove that the class of nonregular languages over $\{0, 1\}$ is not closed under the $SUBSTRING$ operation by giving an example language A that is nonregular but for which $SUBSTRING(A)$ is regular. A complete solution will give (1) a clear and precise description of the language, (2) a justification for why it is nonregular (either by proving this directly or by referring to specific examples from the class or textbook), (3) a description of the result of applying the $SUBSTRING$ operation to the language, and (4) a justification for why this resulting language is regular.
- (c) (*Graded for correctness*) Prove that the class of nonregular languages over $\{0, 1\}$ is not closed under the $EXTEND$ operation by giving an example language B that is nonregular but for which $EXTEND(B)$ is regular. A complete solution will give (1) a clear and precise description of the language, (2) a justification for why it is nonregular (either by proving this directly or by referring to specific examples from the class or textbook), (3) a description of the result of applying the $EXTEND$ operation to the language, and (4) a justification for why this resulting language is regular.
- (d) (*Graded for completeness*) Prove that the class of nonregular languages over $\{0, 1\}$ is not closed under the Kleene star operation by giving an example language C that is nonregular but for which C^* is regular. A complete solution will give (1) a clear and precise description of the language, (2) a justification for why it is nonregular (either by proving this directly or by referring to specific examples from the class or textbook), (3) a description of the result of applying the Kleene star operation to the language, and (4) a justification for why this resulting language is regular.

5. Regular and nonregular languages and Push-down automata (PDA) (6 points): On page 7 of the week 4 notes, we have the following list of languages over the alphabet $\{a, b\}$

$$\begin{aligned} &\{a^n b^n \mid 0 \leq n \leq 5\} \quad \{b^n a^n \mid n \geq 2\} \quad \{a^m b^n \mid 0 \leq m \leq n\} \\ &\{a^m b^n \mid m \geq n + 3, n \geq 0\} \quad \{b^m a^n \mid m \geq 1, n \geq 3\} \\ &\{w \in \{a, b\}^* \mid w = w^R\} \quad \{ww^R \mid w \in \{a, b\}^*\} \end{aligned}$$

- (a) (*Graded for completeness*) Pick one of the regular languages and design a regular expression that describes it and a DFA that recognizes it. Briefly justify your regular expression

by connecting the subexpressions of it to the intended language and referencing relevant definitions. Briefly justify your DFA design by explaining the role each state plays in the machine, as well as a brief justification about how the strings accepted and rejected by the machine connect to the specified language.

- (b) (*Graded for completeness*) Pick one of the nonregular languages and design a PDA that recognizes it. Draw the state diagram of your PDA. Briefly justify your design by explaining the role each state plays in the machine, as well as a brief justification about how the strings accepted and rejected by the machine connect to the specified language.